SortSense: ระบบตรวจจับและแยกประเภทขยะด้วย YOLOv8

รายงานโครงงาน Capstone — Deep Learning

ผู้จัดทำ: [กรอกชื่อ-นามสกุล / รหัสนักศึกษา] | วันที่: 29 ตุลาคม 2026 | Repo: [กรอกลิงก์ GitHub]

---

1. ปัญหาและข้อมูล (Problem & Data)

โจทย์ของโครงงานคือการตรวจจับและจำแนกประเภทขยะจากภาพ/วิดีโอกล้องแบบเรียลไทม์ ออกเป็น 4 ประเภท ได้แก่ Recyclable (รีไซเคิล), Organic (อินทรีย์), General (ทั่วไป) และ Hazardous (อันตราย) เพื่อแนะนำถังขยะที่ถูกต้องให้ผู้ใช้งานแบบอัตโนมัติ งานนี้เป็น object detection ไม่ใช่เพียง image classification เพราะต้องระบุตำแหน่ง (bounding box) ของขยะแต่ละชิ้นในภาพ รองรับหลายวัตถุ ต่อเฟรม และต้องประมวลผลได้เร็วพอสำหรับกล้องสด (webcam / กล้อง iPad)

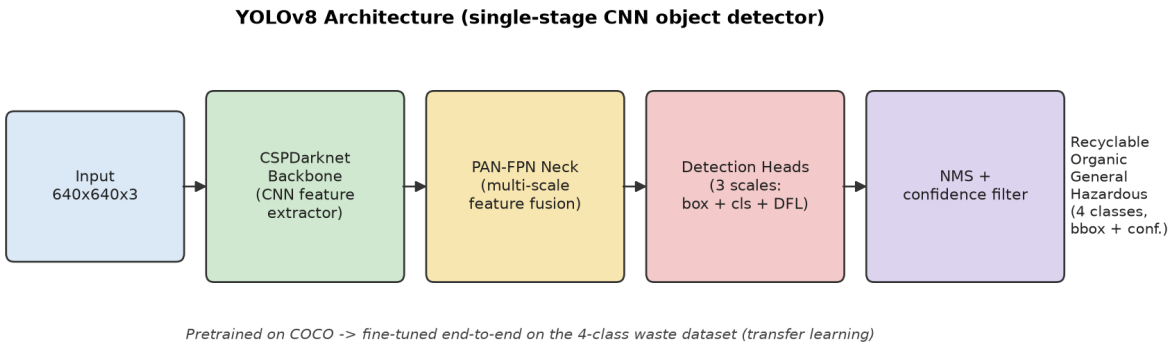
ข้อมูล (Data)

- แหล่งที่มา: ภาพถ่ายขยะจริงที่เก็บและติดป้ายกำกับ (bounding box) เอง ไม่ใช่ชุดข้อมูลสาธารณะ (self-collected & manually annotated)
- ขนาดชุดข้อมูล: 200 ภาพ / 229 bounding boxes (ชุด dataset_waste_real_v3_hazard_single)
- การแบ่งชุดข้อมูล: Train 151 ภาพ / Validation 25 ภาพ / Test 24 ภาพ แบ่งแบบ fixed split ไม่ใช่ random split ใหม่ทุกครั้งที่รัน เพื่อให้ผล reproducible
- ป้องกัน test leakage: ก่อนเทรนทุกครั้งมีขั้นตอนคำนวณ SHA256 hash ของไฟล์ภาพทุกไฟล์ และตรวจว่า ไม่มีภาพใดซ้ำกันข้าม split (ดูขั้นตอน "2. ตรวจภาพและ labels" ใน Kaggle notebook)

ข้อจำกัดสำคัญของข้อมูล: ชุดข้อมูลมีขนาดเล็กมากเมื่อเทียบกับงาน object detection ทั่วไป โดยเฉพาะคลาส Hazardous ที่มีตัวอย่างใน Test เพียง 4 วัตถุ ทำให้ตัวเลขผลลัพธ์ต่อคลาสของ Hazardous มีความแปรปรวนสูงและควรตีความอย่างระมัดระวัง (รายละเอียดเพิ่มเติมในหัวข้อผลลัพธ์และข้อจำกัด)

2. สถาปัตยกรรมโมเดลและการเทรน (Architecture & Training Choices)

โมเดลที่ใช้คือ YOLOv8 (Ultralytics) ซึ่งเป็น CNN แบบ single-stage object detector เลือกใช้ด้วยเหตุผล 3 ข้อ: (1) งานต้องตรวจจับหลายวัตถุพร้อมตำแหน่งในภาพเดียว เหมาะกับสถาปัตยกรรม detector มากกว่า classifier ธรรมดา (2) ต้องประมวลผลแบบเรียลไทม์บนกล้องเว็บ/iPad ซึ่ง YOLO ออกแบบมาเพื่อความเร็วโดยเฉพาะ (3) มี pretrained weights บน COCO ให้ทำ transfer learning ได้ทันที ซึ่งจำเป็นมากเมื่อชุดข้อมูลมีเพียง 200 ภาพ



ภาพที่ 1: สถาปัตยกรรม YOLOv8 — backbone แบบ CNN, neck รวม feature หลายขนาด, detection head 3 scale และ NMS กรองกล่องซ้ำ ก่อนได้ผลลัพธ์ 4 คลาส

ฟังก์ชันสูญเสีย (Loss)

ใช้ loss มาตรฐานของ Ultralytics YOLOv8 ซึ่งรวม 3 ส่วน: box regression loss (CIoU) สำหรับตำแหน่งกรอบ, classification loss (binary cross-entropy) สำหรับจำแนกคลาส และ distribution focal loss (DFL) เพื่อเพิ่มความแม่นยำของขอบกล่อง

Optimizer

ใช้ optimizer='auto' ของ Ultralytics ซึ่งเลือก AdamW โดยอัตโนมัติเมื่อชุดข้อมูลมีขนาดเล็กเช่นนี้ พร้อม weight decay เริ่มต้น (~0.0005) และ cosine learning-rate schedule ค่าที่ใช้จริงถูกบันทึกไว้ใน args.yaml ของผลการเทรนแต่ละรอบบน Kaggle เพื่อให้ตรวจสอบย้อนหลังได้

Regularization

- Transfer learning จาก COCO pretrained weights — ลดความเสี่ยง overfitting กับชุดข้อมูลขนาดเล็ก
- Early stopping: patience=25 epoch (เพดานสูงสุด 100 epoch) หยุดเทรนเมื่อ validation ไม่ดีขึ้น
- Data augmentation ค่าเริ่มต้นของ Ultralytics: mosaic, random HSV color jitter, flip, scale
- Weight decay ในตัว optimizer

Hyperparameters (fixed seed เพื่อ reproducibility)

พารามิเตอร์	ค่า
epochs สูงสุด	100
patience (early stopping)	25
batch size	8
image size	640 x 640
random seed	42 (เท่ากันทุกการเทรน รวมทั้ง baseline และโมเดลหลัก)

3. การประเมินผล (Evaluation)

Metric หลักที่ใช้คือ mAP50 และ mAP50-95 (มาตรฐานของงาน object detection) ร่วมกับ precision และ recall ทั้งแบบรวมและแยกตามคลาส

Baseline

ใช้ YOLOv8n (โมเดลเล็กกว่า พารามิเตอร์น้อยกว่าประมาณ 3 เท่า) เป็น baseline โดยเทรนด้วยชุดข้อมูล, ไฮเปอร์พารามิเตอร์ และ seed เดียวกันทุกประการกับโมเดลหลัก YOLOv8m ต่างกันเฉพาะขนาด backbone เท่านั้น วิธีนี้ทำให้เปรียบเทียบได้ตรงไปตรงมาว่าโมเดลที่ใหญ่และซับซ้อนกว่าให้ผลลัพธ์ดีขึ้นจริงหรือไม่ คุ้มกับต้นทุนการคำนวณที่เพิ่มขึ้นหรือไม่ (สคริปต์ compare_models.py ในโค้ดใช้สร้างตารางเปรียบเทียบนี้)

ป้องกันการรั่วไหลของ Test set (no leakage)

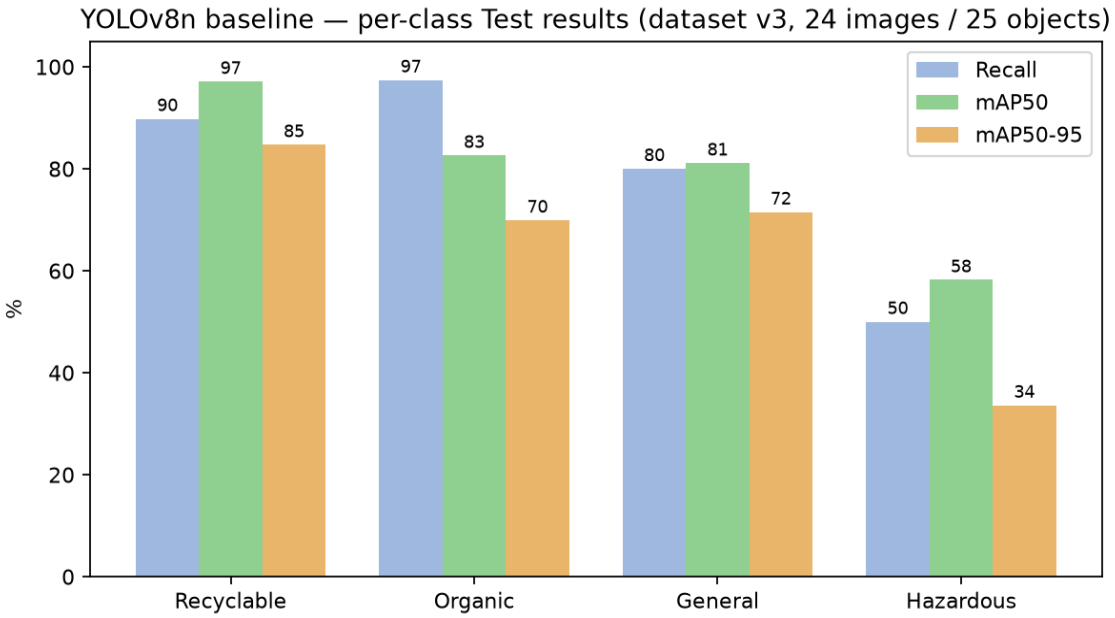
เลือกจุดเทรนที่ดีที่สุดด้วย Validation set เท่านั้น (ใช้กำหนด early stopping และเลือก checkpoint) ส่วน Test set จะถูกแตะเพียงครั้งเดียวหลังเลือกโมเดลเสร็จสิ้นแล้ว ไม่มีการย้อนกลับไปปรับโมเดลตามคะแนน Test

4. ผลลัพธ์ (Results)

ตารางที่ 1 เปรียบเทียบ baseline (YOLOv8n) กับโมเดลหลัก (YOLOv8m) บน Test set เดียวกัน (dataset v3, 24 ภาพ / 25 วัตถุ)

Metric	YOLOv8n (baseline)	YOLOv8m (main model)
Precision	77.3%	[รอผลจาก GPU run]
Recall	79.3%	[รอผลจาก GPU run]
mAP50	79.7%	[รอผลจาก GPU run]
mAP50-95	64.9%	[รอผลจาก GPU run]

[TODO ก่อนส่งจริง] ตัวเลขคอลัมน์ YOLOv8m ยังไม่มีเพราะต้องรันบน GPU — รัน kaggle_train_waste_real_v3_yolov8m.ipynb บน Kaggle/Colab แล้วใช้ compare_models.py สร้างตารางนี้ใหม่ด้วยค่าจริง (ดูขั้นตอนใน README หัวข้อ "Baseline comparison")



ภาพที่ 2: ผลลัพธ์ของ baseline (YOLOv8n) แยกตามคลาสบน Test set — Recyclable แม่นยำสูงสุด ขณะที่ Hazardous ต่ำสุดในทุก metric

Learning curves

[TODO ก่อนส่งจริง] แนบกราฟ train/validation loss และ mAP ต่อ epoch ของทั้งสองโมเดล ซึ่ง Ultralytics สร้างให้อัตโนมัติเป็นไฟล์ results.png ในโฟลเดอร์ output ของ Kaggle (runs/detect/waste_real_v3_hazard_single/results.png และ runs/detect/waste_real_v3_yolov8m/results.png) หลังรันทั้งสอง notebook ให้ดาวน์โหลดไฟล์นี้มาวางแทนย่อหน้านี้

ผลที่ไม่เป็นไปตามคาด (Honest Failure Case)

จุดอ่อนที่ชัดเจนที่สุดของโมเดลคือคลาส Hazardous — mAP50 เพียง 58.2% และ recall เพียง 50% (ตรวจพบวัตถุอันตรายจริงได้เพียงครึ่งหนึ่งของ Test set) สาเหตุที่น่าจะเป็นไปได้มีสองประการ: (1) จำนวนตัวอย่างในชุดข้อมูลน้อยมาก ทั้งหมดรวมกันหลักสิบภาพเท่านั้น และ (2) วัตถุอันตราย (เช่น แบตเตอรีของมีคม) มีรูปร่างและพื้นผิวหลากหลายกว่าขยะรีไซเคิล (ขวด/กระป๋องมีรูปทรงสม่ำเสมอกว่ามาก) ทำให้โมเดลเรียนรู้ pattern ที่สรุปทั่วไปได้ยากกว่าด้วยข้อมูลปริมาณเท่านี้ นอกจากนี้จากการตรวจผลเชิงคุณภาพ

ยังพบกรณีทีโมเดลให้ bounding box ซ้าซ้อนบนวัตถุชิ้นเดียวกัน (เช่น
ขวดพลาสติกหนึ่งขวดถูกล้อมกรอบRecyclable สองกรอบซ้อนกัน) ซึงบ่งชีว่าอาจต้องปรับ NMS (non-max suppression) threshold เพิมเติม

5. ข้อจำกัดและแนวทางพัฒนาต่อ (Limitations & Next Steps)

- ชุดข้อมูลเล็กมาก (200 ภาพ) โดยเฉพาะคลาส Hazardous ทีมีตัวอย่าง Test เพียง 4 ชิน ทำให้ตัวเลขต่อคลาส มีความแปรปรวนสูง ไม่ควรตีความว่าเป็นค่าความแม่นยำระดับใช้งานจริง (production-grade)
- พบกรณี duplicate bounding box จากการตรวจผลเชิงคุณภาพ แต่ยังไม่ได้วัดผลเชิงปริมาณอย่างเป็นระบบ (เช่น อัตรา duplicate detection ต่อภาพ)
- ยังไม่ได้ทดลองปรับความเข้ม augmentation หรือ hyperparameter อืนนอกจากค่าเริมต้นของ Ultralytics
- แนวทางพัฒนาต่อ: (1) เก็บภาพคลาส Hazardous เพิมให้สมดุลกับคลาสอืน (2) ทดลองปรับ confidence/NMS threshold แยกตามคลาส โดยให้ Hazardous มี threshold ต่ำกว่าเพือลด false negative เพราะผลกระทบด้าน ความปลอดภัยสูงกว่าขยะทัวไป (3) เพิมกลไกตรวจทานโดยมนุษย์ (human-in-the-loop) เมือความมันใจของโมเดลต่ำ ก่อนนำไปสังการถังขยะจริง